

Lab 7: Graph Matchings and Applications

Q1. Task Assignment Problem

A company has m employees and n tasks. Each employee is capable of performing only certain tasks. The company wishes to assign as many tasks as possible subject to:

- each employee can be assigned at most one task, and
- each task can be assigned to at most one employee.

Model this as a bipartite graph with employees on the left and tasks on the right; an edge between employee i and task j indicates that employee i can do task j . Your program should compute the *maximum* number of task assignments and may optionally output an explicit assignment.

Input format

- First line: m (number of employees) and n (number of tasks).
- Second line: k , the total number of edges (optional).
- Next m lines: for employee i , first d_i (the number of tasks they can do), followed by d_i integers from $[1, n]$ listing the task indices.

Output format

- First line: the size of the maximum matching (maximum number of tasks assigned).
- Optional: the matching itself, printed as pairs “employee task”.

Sample

Input:

```
4 5
7
2 1 3
3 1 2 5
2 2 4
1 5
```

Possible Output:

4
1 3
2 1
3 2
4 5

Hint

This problem is related to the concept of **maximum bipartite matching**.
Useful references:

- CP-Algorithms: Kuhn's Algorithm for Maximum Bipartite Matching
- Wikipedia: Hall's Marriage Theorem

Q2. Minimum Vertex Cover in Bipartite Graphs

Imagine a company wants to place security cameras in a building with two types of rooms (say, offices A and meeting rooms B). Each camera can monitor all the corridors connected to the room it is installed in. The goal is to place as few cameras as possible so that every corridor is monitored. This is equivalent to finding a minimum vertex cover in the bipartite graph representing rooms and corridors.

Input format

The first line contains two integers $|A|$ and $|B|$, the number of vertices in the two parts. The next line contains an integer E , the number of edges. Each of the next E lines contains two integers u and v indicating an edge between vertex u in A and vertex v in B .

Output format

Print the minimum vertex cover set, listing the vertices from both A and B .

Hint

Use the concept of **vertex cover** to solve this problem.

Note: The vertex cover problem is NP-complete for general graph, but for bipartite graphs it can be solved in polynomial time using König's theorem.

No vertex in a vertex cover can cover more than one edge of M (because overlapping edges would prevent M from being a matching). Therefore, a vertex cover of size $|M|$ is a minimum cover.

To construct such a cover:

- Let U be the set of unmatched vertices in A .
- Let Z be the set of vertices that are either in U or connected to U by alternating paths (paths that alternate between edges in the matching and edges not in the matching).
- Then the minimum vertex cover is $K = (A \setminus Z) \cup (B \cap Z)$.

Reference: König's theorem (Wikipedia)

Q3. Hungarian Algorithm for the Assignment Problem

Consider a company with n workers and n jobs. Each worker specifies the cost they expect for performing each job. Each worker can be assigned to exactly one job, and each job must be assigned to exactly one worker. The goal is to assign jobs to workers to minimize the total cost. This problem can also be formulated as:

- Given an $n \times n$ cost matrix A , select exactly one number from each row and each column so that the sum of the selected numbers is minimized.
- Equivalently, find a permutation p of length n that minimizes $\sum_i A[i][p[i]]$.
- In graph terms, consider a complete bipartite graph with n vertices in each part, with edge weights representing costs. Find a perfect matching with minimum total weight.

Input format

The first line contains an integer n , the number of workers and jobs. The next n lines contain n integers each, representing the cost matrix A .

Output format

Print the minimum total cost for assigning all jobs to workers.

Hint

Use the **Hungarian Algorithm** (a.k.a. Kuhn-Munkres algorithm) to solve this problem efficiently.

References:

- CP-Algorithms: Hungarian Algorithm
- TopCoder: Assignment Problem and Hungarian Algorithm

Homework: Maximum Matching in General Graphs via Edmonds' Algorithm

Problem Statement

Given a general (not necessarily bipartite) graph $G = (V, E)$, find a maximum matching — a largest possible set of edges such that no two edges share a vertex.

Motivation

Imagine you are forming pairs in a social network, where each person can only be in one pair. Unlike bipartite cases (such as workers and jobs), here any person can be paired with any other. This leads to the general matching problem, which is harder to solve.

Input Format

- The first line contains two integers n and m — the number of vertices and edges.
- Each of the next m lines contains two integers u, v representing an edge (u, v) .

Sample Output

For example, if a matching of size 3 is found, then output could be:

```
Maximum matching size: 3
```

Hint

Use Edmonds' Blossom Algorithm for maximum matching in general graphs.

Reference

<https://codeforces.com/blog/entry/92339>